1 Methodology: End-to-End Narrative Shift Detection Pipeline

This section presents a complete, step-by-step description of the proposed narrative shift detection pipeline. Each stage is explained in terms of its motivation, implementation, input and output formats, and design justification. The goal is to detect how narratives evolve over time, even when entities change, without relying on labeled data.

1.1 Overall Objective

The objective of this work is to detect **narrative shifts**, defined as changes in how a topic is framed or discussed over time. Unlike topic change detection, narrative shift detection focuses on semantic framing rather than surface-level entity overlap. The proposed framework is designed to operate:

- at large scale,
- without supervision,
- across multiple topics,
- and under evolving entities.

1.2 Running Example (carried through stages)

To make the pipeline explicit we use a *single running example* that will be passed through every stage.

Article (single-row corpus example):

> Date: 2022-06-15
> Text: "Artificial intelligence is rapidly expanding. Data centers now consume massive amounts of electricity. Governments are beginning to regulate AI usage."

We will show how this single article produces sentence-level examples, embeddings, topic weights, aggregated daily states and a toy narrative-shift score for illustration.

1.3 Stage 0: Raw Dataset

Input: A raw corpus of news articles represented as a table with two columns:

- `date`: publication date
- `article_text`: full article content

Example Input:

> Date: 2022-06-15
> Text: "Artificial intelligence is rapidly expanding. Data centers now consume massive amounts of electricity. Governments are beginning to regulate AI usage."

Motivation: Articles are long, may contain multiple topics, and often mix several narrative frames. Narrative shifts occur at a finer granularity than the article level, motivating further decomposition.

1.4 Stage 1: Sentence Segmentation

What we do: Each article is segmented into sentences while preserving their original order.

Why: Narrative framing changes at the sentence level. Using full articles obscures fine-grained narrative transitions.

How: Standard sentence tokenization (e.g., spaCy or NLTK).

Output Format:

sentence_id	article_id	date	sentence_text
s1	A1	2022-06-15	Artificial intelligence is rapidly expanding.
s2	A1	2022-06-15	Data centers now consume massive amounts of electricity.
s3	A1	2022-06-15	Governments are beginning to regulate AI usage.

Running-Example Note: The single article yields three ordered sentences (s1, s2, s3). We keep their order since temporal context and sentence neighbors will be used in subsequent stages.

Important Clarification: Sentence segmentation alone does not discard article-level meaning. Context is explicitly reintroduced in the next stage.

1.5 Stage 2: Context-Aware Sentence Embedding

Problem: Isolated sentences can be semantically ambiguous. For example, a sentence about energy consumption may appear climate-related unless contextualized.

Solution: Each sentence is embedded together with its immediate neighbors.

Method: For a sentence s_i, we construct a contextual input:

$$(s_{i-1}, s_i, s_{i+1})$$

and encode it using Sentence-BERT (SBERT).

Example (contextual input for each sentence):

- For **s1**: contextual triple = (*¡START¿*, s1, s2) = (””, ”Artificial intelligence is rapidly expanding.”, ”Data centers now consume massive amounts of electricity.”)
- For **s2**: contextual triple = (s1, s2, s3)
- For **s3**: contextual triple = (s2, s3, *¡END¿*)

Example (illustrative/truncated embeddings): We show a truncated toy illustration (real SBERT embeddings are 768-dimensional):

$$\mathbf{s}_1^{\text{ctx}} \approx [0.60,\ 0.12,\ -0.03,\ \dots]$$
$$\mathbf{s}_2^{\text{ctx}} \approx [0.32,\ 0.45,\ 0.01,\ \dots]$$
$$\mathbf{s}_3^{\text{ctx}} \approx [0.58,\ 0.10,\ -0.02,\ \dots]$$

These vectors are understood as *contextual* sentence encodings (neighbors included).

Why SBERT:

- Captures semantic meaning beyond keywords
- Robust to entity changes
- Efficient and scalable

- Widely validated in NLP research

Output: A dense semantic vector (e.g., 768 dimensions) per sentence.

Key Benefit: Article integrity is preserved at the representation level while enabling sentence-level analysis.

1.6 Stage 3: Topic Embedding Construction

Motivation and Intuition:

In this work, a *topic* is not treated as a discrete label (e.g., "Technology = class 3"). Instead, a topic is viewed as a *region of meaning* in a continuous semantic space. This distinction is critical for narrative analysis, where the same topic may be expressed using different entities, terminology, or indirect references over time.

Sentence-BERT (SBERT) does not operate on symbolic labels. Rather, it encodes *patterns of meaning* expressed through words, phrases, and sentences. Therefore, instead of assigning topics, we *describe* them using representative semantic text.

Key Concept: A topic embedding is not a label, rule, or keyword list. Instead, it represents the *center of gravity of meaning* associated with that topic.

Topic Description Construction:

For each topic, we construct a short natural-language description consisting of phrases or sentences that characterize the semantic scope of the topic. These descriptions need not match article sentences exactly; their purpose is to define the semantic boundary of the topic.

Example Topic Descriptions:

- **Technology**: "Artificial intelligence, machine learning, software systems, automation, data centers, computing infrastructure."
- **Climate**: "Environment, pollution, sustainability, carbon emissions, global warming, energy usage."

Embedding Procedure:

Each topic description is encoded using the *same SBERT encoder* used for sentence embeddings. This ensures that topic representations and sentence representations reside in the same semantic embedding space.

Formally, for a topic k, its embedding is defined as:

$$\mathbf{T}_k \in \mathbb{R}^{768}$$

where $\mathbf{T}_k$ is the SBERT embedding of the topic description text.

Example (toy/truncated topic embeddings):

$$\mathbf{T}_{\text{Technology}} \approx [0.65,\ 0.08,\ -0.01,\ \ldots]$$
$$\mathbf{T}_{\text{Climate}} \approx [0.20,\ 0.55,\ 0.10,\ \ldots]$$

Why Shared Embedding Space Matters:

Encoding both sentences and topics using the same model allows direct semantic comparison via similarity measures. As a result, sentence-level topic relevance can be computed without supervision, even when entities differ or topics overlap.

Output:

The output of this stage is a set of topic embeddings:

$$\{\mathbf{T}_1, \mathbf{T}_2, \ldots, \mathbf{T}_K\}$$

where each embedding represents the semantic center of a topic.

1.7 Stage 4: Sentence-Level Topic Weighting

What we do: We compute the cosine similarity between each sentence embedding and each topic embedding:

$$w_i^{(k)} = \cos(\mathbf{s}_i, \mathbf{T}_k)$$

Example (toy cosine similarities for each sentence): Using the running-example contextual sentence vectors and the topic embeddings above (truncated for display):

- **s1** ("Artificial intelligence is rapidly expanding."):
  - Technology: $w_1^{(\text{Tech})} \approx 0.93$
  - Climate: $w_1^{(\text{Climate})} \approx 0.05$
- **s2** ("Data centers now consume massive amounts of electricity."):
  - Technology: $w_2^{(\text{Tech})} \approx 0.65$
  - Climate: $w_2^{(\text{Climate})} \approx 0.40$
- **s3** ("Governments are beginning to regulate AI usage."):
  - Technology: $w_3^{(\text{Tech})} \approx 0.85$
  - Climate: $w_3^{(\text{Climate})} \approx 0.12$

Why Soft Assignment:

- Preserves ambiguity
- Avoids forcing incorrect labels
- Essential for overlapping narratives

1.8 Stage 5: Topic Filtering and Temporal Ordering

Filtering: For each topic, we retain sentences whose topic weight exceeds a predefined threshold (e.g., $\tau = 0.5$).

Example (filtering with threshold $\tau = 0.5$):

- **Technology**: retain s1 (0.93), s2 (0.65), s3 (0.85) – all three pass the threshold.
- **Climate**: only s2 has moderate weight (0.40) and is *not* retained when $\tau = 0.5$.

Temporal Ordering: All retained sentences are sorted by date in non-decreasing order.
Important Rule:

- Sentences may come from different articles
- Temporal order is always preserved
- No shuffling or temporal jumping is performed

1.9 Stage 6: Handling Same-Date Articles via Temporal Aggregation

Motivation: Narrative shifts rarely occur within a single day. Multiple articles published on the same date typically reflect diverse viewpoints rather than temporal evolution. If treated independently, same-day sentences may introduce spurious narrative shifts.

Key Idea: Each date is treated as a single narrative state for a given topic.

Method: For a fixed topic k and date d, all contextual sentence embeddings $\{\mathbf{s}_1, \mathbf{s}_2, \ldots, \mathbf{s}_{N_d}\}$ published on that date are aggregated into a single representation:

$$Z_{k,d} = \frac{1}{N_d} \sum_{i=1}^{N_d} \mathbf{s}_i$$

Example (aggregation for Technology on 2022-06-15): All three retained Technology sentence embeddings are averaged:

$$Z_{\text{Tech, } 2022-06-15} = \frac{1}{3} \left(\mathbf{s}_1^{\text{ctx}} + \mathbf{s}_2^{\text{ctx}} + \mathbf{s}_3^{\text{ctx}} \right)$$

This yields a single daily narrative vector representing how Technology was framed on 2022-06-15.

Important Clarification: No temporal windowing is performed at this stage. The aggregation window is defined solely by the calendar date.

Why Mean Pooling:

- Produces a stable summary of same-day narrative framing
- Robust to article volume fluctuations
- Preserves semantic signal without introducing bias

Output: A single narrative representation per topic per date, which serves as the input to temporal window construction in the subsequent stage.

1.10 Stage 7: Topic-Wise Adaptive Temporal Windowing

Motivation:

Narratives evolve at different temporal rates depending on the topic. For instance, technology-related narratives (e.g., artificial intelligence) can shift within days due to rapid innovation or regulation, whereas health or climate narratives typically evolve over weeks or months. Using a fixed temporal window for all topics therefore leads to either missed shifts or spurious noise.

Key Principle: Narrative time is topic-dependent. Accordingly, temporal window size and stride must be adapted per topic.

Approach:

For each topic k, we define:

- a topic-specific window size W_k
- a topic-specific stride S_k

These parameters control the temporal span of each window and the degree of overlap between consecutive windows.

Example Configuration:

- **Technology (AI)**: $W_k = 3$–5 days, $S_k = 1$ day

- **Health**: $W_k = 21$–30 days, $S_k = 7$ days
- **Climate**: $W_k = 30$–60 days, $S_k = 14$ days

Window Construction:

Let $\{Z_1, Z_2, \ldots, Z_T\}$ denote the aggregated daily narrative representations for a given topic, sorted in non-decreasing temporal order. Continuous sliding windows are constructed as:

$$[Z_t, Z_{t+1}, \ldots, Z_{t+W_k-1}]$$

with stride S_k.

Example (toy window construction around our example date): Suppose we have daily aggregated Technology vectors for 2022-06-13 to 2022-06-17:

- $Z_{2022-06-13}, Z_{2022-06-14}, Z_{2022-06-15}, Z_{2022-06-16}, Z_{2022-06-17}$

With $W_k = 3$ and $S_k = 1$, the windows are:

- $W_1 = [Z_{2022-06-13}, Z_{2022-06-14}, Z_{2022-06-15}]$
- $W_2 = [Z_{2022-06-14}, Z_{2022-06-15}, Z_{2022-06-16}]$
- $W_3 = [Z_{2022-06-15}, Z_{2022-06-16}, Z_{2022-06-17}]$

Each window is then summarized (e.g., mean or an RNN/transformer encoder) to produce a window representation used for temporal contrastive learning.

Design Constraints:

- Windows are temporally contiguous
- No skipping or non-adjacent jumps are permitted
- Overlapping windows preserve narrative continuity

Benefit:

Topic-wise adaptive windowing ensures that fast-evolving narratives are captured without smoothing, while slow-evolving narratives are modeled robustly without overreacting to short-term fluctuations.

1.11 Stage 8: Temporal Contrastive Learning (TCL)

Objective: The goal of Temporal Contrastive Learning (TCL) is to learn representations that distinguish narrative stability from narrative change over time, without requiring labeled shift annotations.

Why Contrastive Learning: Narrative shift is inherently a relative phenomenon. Rather than assigning absolute labels, TCL learns from temporal relationships: what remains stable across nearby time windows and what diverges across distant ones.

Method: Given a sequence of temporally ordered narrative windows $\{W_1, W_2, \ldots, W_T\}$, TCL constructs training pairs as follows:

- **Positive pairs:** temporally adjacent windows (W_t, W_{t+1})
- **Negative pairs:** temporally distant windows $(W_t, W_{t+k}),\ k \gg 1$

The model is trained to minimize the distance between positive pairs and maximize the distance between negative pairs in representation space.

Example (positive/negative pairs using toy windows): Using the windows from Stage 7:

- Positive pair: (W_1, W_2) where $W_1 = [Z_{2022-06-13}, Z_{2022-06-14}, Z_{2022-06-15}]$ and $W_2 = [Z_{2022-06-14}, Z_{2022-06-15}$
- Negative pair: (W_1, W_5) where W_5 is far in the future (e.g., 30 days later).

Why TCL is Chosen:

- Does not require labeled narrative shift data
- Naturally models gradual temporal evolution
- Robust to noise and entity changes
- Learns temporal structure rather than static similarity

Why Alternative Methods Fail:

- Topic models conflate topic presence with narrative framing
- Distance-based heuristics lack temporal learning and threshold stability
- Clustering methods fail to capture gradual narrative drift
- Supervised methods require unavailable and subjective annotations
- Generative approaches lack reliability and interpretability

Output: The output of TCL is a temporally-aware narrative embedding Z_t for each time window, encoding both semantic content and its temporal position.

1.12 Stage 9: Narrative Shift Scoring

Narrative shift magnitude is computed as:

$$\Delta_t = \|Z_t - Z_{t-1}\|$$

Example (toy numeric illustration): Assume the learned window representations around mid-June produce the following (toy) distances:

- $\Delta_{2022-06-15} = 0.12$ (minor change)
- $\Delta_{2022-06-16} = 0.18$ (small drift)
- $\Delta_{2022-06-18} = 0.72$ (large shift — e.g., introduction of new regulatory framing)

A value like 0.72 would be flagged for analyst inspection as a significant narrative transition.

1.13 Stage 10: Sentence-Level Explanation

We identify sentences contributing most to representation changes between adjacent windows. This provides transparent and reviewer-friendly explanations.

Example (explanation for a large shift on 2022-06-18):

Compute contribution scores by measuring how much removing a sentence would reduce the change magnitude (ablation) or by ranking sentences by change in their topic-similarity delta.

In our running example, the sentence most responsible for the jump from a technical to a regulatory framing is:

> "Governments are beginning to regulate AI usage."

This sentence increases the relative weight of regulatory framing and thus explains the large Δ observed. In practice we return the top-k contributing sentences with provenance (article id, date).

1.14 Stage 11: Post-Training Validation and Entity-Centric Narrative Analysis

Purpose: This stage enables practical validation and downstream use of the trained narrative shift model. Given a user-specified topic and entity, the system detects how narratives surrounding that entity evolve over time within the selected topic.

1.14.1 User Input Specification

After training is completed, the user provides:

- **Topic** T: one of the predefined semantic topics (e.g., Technology, Climate).
- **Entity** E: a named entity or concept of interest (e.g., "OpenAI", "China", "Electric Vehicles").
- **Article Set**: a collection of news articles with publication dates.

Input Format Example:

Topic: Technology
Entity: Artificial Intelligence
Articles: $\{(\mathrm{d}_1, a_1), (d_2, a_2), \ldots, (d_n, a_n)\}$

1.14.2 Entity-Aware Sentence Selection

From the provided articles, we reuse the trained pipeline components:

- Sentence segmentation
- Context-aware SBERT sentence embedding
- Topic relevance scoring

We then retain sentences that satisfy:

- High semantic relevance to topic T, and
- Either explicit mention of entity E or high semantic similarity to the entity embedding.

Entity Embedding: The entity name or description is embedded using SBERT to enable semantic matching beyond exact string mentions.

Example (entity selection using the running example): Entity: "Artificial Intelligence". Sentences retained (from 2022-06-15): s1 and s3 (both explicitly refer to AI concept or usage). s2 is retained for contextual signals where relevant.

1.14.3 Temporal Aggregation Around the Entity

For each date d, all entity-relevant sentences are aggregated to form a daily entity-centric narrative state:

$$Z_d^{(E,T)} = \text{Aggregate}\left(\{\mathbf{s}_i \mid s_i \text{ relevant to } E \text{ and } T \text{ on date } d\}\right)$$

This representation captures how the entity is framed on a given day within the chosen topic.

1.14.4 Narrative Shift Detection

Using the trained temporal contrastive representation space, we compute narrative shift scores across consecutive time units:

$$\Delta_d^{(E,T)} = \| Z_d^{(E,T)} - Z_{d-1}^{(E,T)} \|$$

Interpretation:

- Low score: stable narrative framing around the entity
- High score: significant narrative shift (e.g., praise to criticism, technical to regulatory framing)

Example (entity-centric output for "Artificial Intelligence"):

Date	Narrative Shift Score	Key Sentence
2022-06-15	0.05	"Artificial intelligence is rapidly expanding."
2022-06-16	0.10	"Data centers now consume massive amounts of electricity."
2022-06-18	0.72	"Governments are now imposing strict regulations on AI energy consumption."

1.14.5 Output Format

The system returns:

- A time series of narrative shift scores
- Key sentences responsible for major shifts
- Corresponding dates and articles

Output Example:

Date: 2022-06-18
Narrative Shift Score: 0.72
Key Sentence: "Governments are now imposing strict regulations on AI energy consumption."

Why This Validation Is Sound

- No retraining or fine-tuning is performed
- The model generalizes to unseen entities
- Entity-specific analysis emerges from semantic filtering, not supervision

This design enables controlled, interpretable validation while preserving the unsupervised and topic-agnostic nature of the proposed framework.

1.15 Final Summary

The pipeline preserves article meaning through contextual sentence embeddings, applies soft topic weighting to handle overlap, aggregates same-date content into unified narrative states, constructs continuous temporal windows, and employs temporal contrastive learning to model narrative evolution. Narrative shifts are detected as representation divergence across adjacent time windows.

1.16 Final Summary

We preserve article meaning through contextual sentence embeddings, apply soft topic weighting to handle overlap, aggregate same-date content into unified narrative states, construct continuous temporal windows, and use temporal contrastive learning to model narrative evolution. Narrative shifts are detected as representation divergence across adjacent time windows.

2 Justification for Temporal Contrastive Learning

2.1 Why Temporal Contrastive Learning is Suitable

Narrative shift detection is fundamentally a temporal representation learning problem. Unlike topic classification or event detection, narrative shifts do not have explicit labels, fixed boundaries, or universally agreed definitions. Instead, narratives evolve gradually over time, often under changing entities and vocabulary.

Temporal Contrastive Learning (TCL) is particularly well-suited for this setting because it learns representations based on *relative temporal relationships* rather than absolute labels. By treating temporally adjacent segments as positive pairs and temporally distant segments as negative pairs, TCL directly models the notion of narrative continuity versus narrative change.

This principle has been shown to be effective across multiple domains involving unlabeled sequential data. For example, TS-TCC [**?**] demonstrates that temporal contrastive objectives can learn robust representations for time-series data by exploiting temporal adjacency. Similarly, dynamic contrastive learning frameworks for time series [**?**] show that temporal contrastive objectives capture long-term evolution patterns without supervision.

Importantly, contrastive learning has also been applied to temporal and narrative text tasks. The ConTempo framework [**?**] applies temporally contrastive objectives to narrative event relations in text, demonstrating that temporal contrast is effective beyond continuous signals. These works collectively support the use of TCL for modeling narrative evolution in news text.

2.2 Why Alternative Methods are Insufficient

Several alternative approaches exist for modeling textual change over time; however, they are not well-suited for generalized narrative shift detection.

Topic Models. Latent Dirichlet Allocation (LDA) and Dynamic Topic Models (DTM) focus on identifying topic presence based on word distributions. While useful for topic discovery, they conflate topic occurrence with narrative framing and are highly sensitive to vocabulary and entity changes. As a result, they fail to capture framing shifts within the same topic.

Distance-Based Heuristics. Methods based on cosine distance or KL divergence between representations measure surface-level difference but do not learn temporal structure. They require manually chosen thresholds and cannot distinguish noise from meaningful narrative change.

Clustering-Based Methods. Clustering approaches group similar texts but lack temporal directionality. Clusters may shift abruptly due to data variation rather than genuine narrative evolution, making them unreliable for gradual change detection.

Supervised Change Detection. Supervised approaches require labeled narrative shift data, which is both scarce and subjective. Narrative interpretation varies across annotators and domains, limiting scalability and generalization.

Generative and LLM-Based Methods. Generative approaches, including large language models, often lack reproducibility, are difficult to evaluate quantitatively, and may introduce hallucinated explanations. These properties make them unsuitable for reliable narrative shift detection in large-scale studies.

2.3 Summary

Temporal Contrastive Learning addresses the core challenges of narrative shift detection: it is unsupervised, robust to entity change, models gradual temporal evolution, and learns directly from temporal structure. In contrast to static similarity measures, topic models, and supervised approaches, TCL provides a principled and scalable solution for modeling narrative dynamics over time.

References (Key Supporting Works)

- TS-TCC: Time-Series Representation Learning via Temporal and Contextual Contrasting. https://arxiv.org/abs/2106.14
- Dynamic Contrastive Learning for Time Series Representation. https://arxiv.org/abs/2410.15416
- ConTempo: A Unified Temporally Contrastive Framework for Temporal Relation Extraction. https://aclanthology.org/2 acl.89/
- TCLR: Temporal Contrastive Learning for Video Representation. https://www.sciencedirect.com/science/article/pii/S1